Virtual Ad Insertion in Tennis Broadcasts

A Region-of-Interest Tracking and Compositing Pipeline

Raghav Punnam Enrique Díaz de León Hicks Giovanni Visi
Martina Maiorana

May 2026

Abstract

We present an end-to-end software pipeline for inserting virtual advertising into live tennis broadcasts. The system replaces baked-in courtside ads with arbitrary sponsor logos while preserving photographic realism through camera motion and through player occlusion (specifically, when a player physically walks across a court-floor logo). The canonical test case places five virtual advertisements simultaneously (three back-wall banners, one left-side banner, and one court-floor walkover logo) onto a thirteen-second sequence from the 2026 Melbourne broadcast in which a player crosses the field of view.

We combine three foundation models (SAM 2 for region segmentation, MatAnyone for video alpha matting, and a TrackNet-derived court-keypoint detector) with two project-specific components: a hybrid-lock state machine that holds the placement pixel-locked to a manually-clicked seed when the camera is still and ramps to a dynamic estimate when motion exceeds a calibrated tolerance, and a custom inpaint-plus-LED-blend compositor that re-bakes the inserted logo's luminance against the local surface response.

We also describe a three-layer evaluation framework that combines deterministic per-region numerical gates, a structured visual rubric scored against paired original-vs-composite crop strips, and direct human visual review against the original broadcast. We report a case where the rubric and the numerical gates agreed on a candidate that visual review ultimately rejected, and use this to motivate the three-layer design. The final delivered pipeline passes every per-region quality gate, achieves a walkover-window occlusion intersection-over-union of 0.985 (gate > 0.80), and maintains temporal structural similarity above 0.99 for every placement region.

Table of contents

1 Introduction

1.1 Motivation

Live sports broadcasts are a primary inventory channel for sponsor advertising: courtside boards, back-wall banners, painted floor logos, scoreboard placements, and umpire-stand signage are all monetised. In current practice these placements are physically baked into the venue and therefore identical for every viewer. *Virtual ad insertion* replaces those baked-in ads with software-rendered logos at broadcast time. The commercial opportunity is large because it permits regional, demographic, and ultimately personalised sponsor targeting on the same physical broadcast, but the technical bar is correspondingly high: the viewer must not be able to tell that the ad has been swapped. The inserted logo must track the camera, occlude correctly when players move in front of it, and look photographically integrated with the surface it is painted on.

Several commercial systems already exist for this task (Supponor for ice-hockey dasherboards [1], uniqFEED for general sports broadcasts [2], and Vizrt's Viz Arena for augmented-reality graphics [3]), but they typically rely on infrared markers, custom camera-rig instrumentation, or trained operator-in-the-loop tools. Our project asks whether the same effect can be achieved purely in software, with off-the-shelf foundation models and a minimal operator-input step (a few clicks on a single seed frame).

1.2 Contributions

This paper makes three contributions.

1. **An end-to-end pipeline** combining SAM 2 [4], a TrackNet-derived court-keypoint detector [5], [6], a hybrid-lock state machine for homography stabilisation, MatAnyone [7] video alpha matting for player occlusion, and a custom inpaint-plus-LED-blend compositor.
2. **A three-layer evaluation framework** with deterministic numerical gates, a structured visual rubric scored against paired original-vs-composite crop strips, and a direct human visual-review gate that overrides the lower layers when they disagree.
3. **A documented project journey** that arrived at the final design through three iteration phases, including one failed experimental axis whose negative result narrowed the design space, plus an instance in which the rubric and the numerical gates jointly favoured a candidate that direct visual review rejected.

1.3 Paper structure

Section 2 frames the placement problem and introduces the Melbourne walkover demonstration clip. Section 3 positions our system against the commercial and academic landscape. Section 4 describes the production architecture component by component, with the relevant equations for each stage. Section 5 presents the three-layer evaluation framework. Section 6 reports the three iteration phases that produced the final design. Section 7 presents per-region quantitative metrics and visual artefacts. Section 8, Section 9, and Section 10 close the paper. The implementation is available in our companion code repository [1].

[1] Implementation: `https://github.com/enriquedlh97/homography-fitting`.

2 Problem Formulation and Demonstration Clip

2.1 What makes virtual ad insertion hard

A naive baseline (alpha-composite a logo image at a fixed quadrilateral on every frame) fails on three independent counts.

Geometry. The broadcast camera is a real pan-tilt-zoom (PTZ) rig with non-zero motion. Logos placed on the court floor or on a sideline banner must stay anchored to those physical surfaces, or the placement visibly drifts off the court. Formally, we require a per-frame homography $H_t \in \mathrm{PGL}(3, \mathbb{R})$ (a 3×3 projective transformation defined up to non-zero scale) that maps the canonical court plane $\mathcal{C} \subset \mathbb{R}^2$ to the broadcast image plane $\mathcal{I}_t \subset \mathbb{R}^2$ for every frame index t.

Occlusion. When a player walks over a court-floor logo, the player's silhouette must occlude the logo at the pixel level (not a bounding box, not a soft alpha sheen, but the actual person shape, including legs, feet, racket, and shadow boundaries). This requires a continuous-valued alpha matte $\alpha_t \in [0, 1]^{H \times W}$ rather than a binary mask.

Photometric realism. The inserted logo must read as if it were physically painted (for the floor) or printed (for the banners). It cannot look pasted on. This implies matching the surface's local brightness response, suppressing visible alpha-feathering edges, and producing a clean inpaint of the original underlying ad.

2.2 The Melbourne walkover demonstration clip

Our canonical test case is `melbourne-walking-over-logo.mov`, a thirteen-second sequence from the 2026 Melbourne broadcast at approximately 59 fps (767 frames at 1920×1080). It packs every hard mode into one clip:

- **Five simultaneous placements**: three back-wall banners (objects 1, 2, 5 in our configuration, initially KIA-baked), one left-side banner (object 4, initially YoPRO-baked), and one court-floor walkover logo (object 3, the baked MELBOURNE wordmark).
- **A player walkover** between approximately frames 685 and 723: a player enters the floor-logo region, walks directly across it, and exits. This is the demanding occlusion case.
- **Mostly static camera with subtle drift and a clearly visible motion segment during the walkover.** This is the exact failure mode that makes a statically-locked placement insufficient.

Table 1 summarises the five regions and the quality constraint that dominates each.

Table 1: Five placement regions on the demonstration clip and the quality constraint that dominates each.

Region	Surface	Dominant constraint
Back banners (×3)	banner	Geometric stability; consistent inpaint; no luminance flicker
Left side banner	banner	Edge realism (no halo at letter edges); texture match
Court floor logo	court_floor	Halo absence; correct player occlusion; logo visibility during walkover

3 Related Work

3.1 Commercial systems

Commercial virtual-ad-insertion products fall into two broad families.

Hardware-instrumented systems like Supponor [1] use infrared-emitting dasherboards plus custom keying hardware for ice-hockey broadcasts. Quality is high and the system is real-time, but every venue requires proprietary instrumentation and a trained operator hub. Vizrt's Viz Arena [3] requires camera-rig instrumentation but offers a richer authoring tool for graphics.

Software-only systems like uniqFEED [2] use custom computer-vision pipelines (typically Hough-line court detection plus hand-tuned trackers). Quality is broadcast-grade but requires per-court tuning and trained operators in the loop. Most academic systems sit in this family conceptually, but at lower realism.

3.2 Academic prior work

Academic interest in sports-broadcast ad placement has been concentrated around soccer-field registration [8], [9]. Both papers focus on the homography-estimation step (registering the broadcast image plane to a canonical pitch) but neither integrates the full ad-replacement pipeline (segmentation, occlusion handling, and photometric blending). TrackNet [5], whose architecture we adapt for our court-keypoint detector, was originally formulated for tennis-ball tracking, but its heatmap-regression backbone has been repurposed by the open-source `TennisCourtDetector` project [6] to localise court corners.

Foundation-model segmentation has matured rapidly in the last two years. SAM [10] established the promptable-segmentation paradigm; SAM 2 [4] extended it to video via a memory-attention module. Video alpha matting, in turn, was elevated by MatAnyone [7], which combines target-assigned recurrent propagation with a region-adaptive memory fusion that preserves continuous alpha values across frames.

3.3 The gap we address

To our knowledge, no published system combines foundation-model segmentation (SAM 2), foundation-model alpha matting (MatAnyone), and a learned-keypoint court estimator (TrackNet-derived) into a single end-to-end ad-replacement pipeline. Our system targets the same effect as hardware-instrumented commercial solutions with two cost reductions: zero in-venue instrumentation and a one-click-per-region operator input. The final pipeline is not yet at the per-frame latency a live broadcast demands (we measure 2.68 fps on a single NVIDIA H200), but it is broadcast-credible on static-camera portions and the per-frame design admits throughput optimisation as an independent engineering axis (see Section 9).

4 Pipeline Architecture

4.1 Overview

The pipeline is a single-pass, per-frame system. Figure 1 summarises the data flow.

The six stages (segmentation, quad fitting, court-geometry estimation, hybrid-lock gating, alpha matting, and compositing) are described in turn.

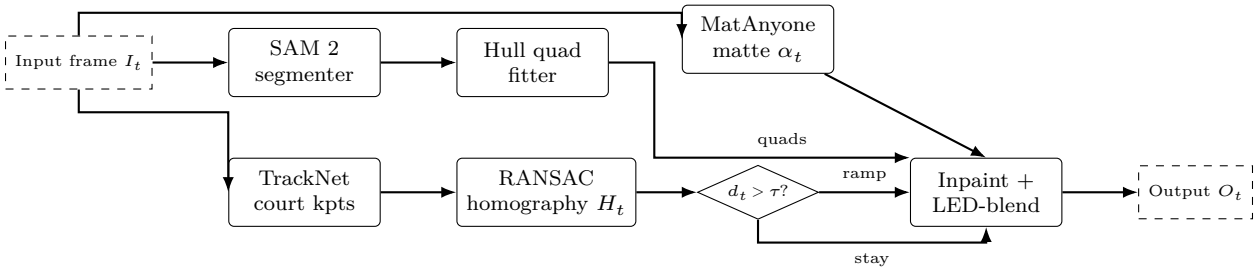

Figure 1: End-to-end pipeline data flow. Per frame: SAM 2 produces banner-region masks; TrackNet-derived keypoint regression yields fourteen court landmarks; RANSAC fits a homography H_t; the hybrid-lock state machine decides whether to ramp toward H_t or stay at the manually-clicked seed. MatAnyone produces a per-frame alpha matte α_t. The compositor erases the original ad, warps the new logo through the chosen homography, applies LED-style brightness re-baking, and occludes through α_t.

4.2 SAM 2 segmentation

We use SAM 2 [4] (Hiera-tiny checkpoint) in image-prompt mode on a single seed frame, with one to three positive clicks per ad region. The resulting per-region binary masks are propagated across all 767 frames via the SAM 2 video tracker, which uses memory attention to stay locked on each region without re-clicking.

SAM 2 is a foundation model for *promptable visual segmentation* that unifies image and video segmentation under a single streaming architecture. A *prompt* is any sparse spatial cue indicating the object of interest (a click, a bounding box, or a coarse mask) which the user supplies on one or more frames. The model returns a pixel-accurate mask and, in the video case, propagates that mask through every subsequent frame.

The per-frame image branch follows the SAM lineage but replaces the original Vision Transformer with a *Hiera* hierarchical vision encoder, whose multi-scale features feed a lightweight prompt encoder and a transformer-based mask decoder. The decoder emits multiple candidate masks together with a scalar intersection-over-union (IoU) prediction used for ambiguity resolution and mask ranking.

The principal departure from SAM 1 is the introduction of a *memory module* that turns the per-image segmenter into a video tracker. Each processed frame is encoded into a compact set of spatial memory tokens and object pointers, written to a first-in-first-out memory bank of recent and prompted frames. When segmenting frame t, the mask decoder is conditioned on the current frame features and on cross-attention against this bank. The memory-attention block of [4] stacks L transformer layers, each performing self-attention followed by cross-attention against the concatenated memory; schematically, with F_t the Hiera image embedding of frame t, M the spatial memory tokens, and P the object pointers, a single block computes

$$F'_t \;=\; F_t \;+\; \mathrm{SelfAttn}(F_t), \qquad \tilde{F}_t \;=\; F'_t \;+\; \mathrm{CrossAttn}(Q{=}F'_t W_Q,\; K{=}[M;P]W_K,\; V{=}[M;P]W_V). \tag{1}$$

The decoder then attends $\tilde{F}_t$ against the prompt tokens to emit the frame-t mask. For each prompt the decoder produces $N{=}3$ candidate masks $\{\hat{m}_t^{(k)}\}_{k=1}^{3}$ with predicted IoU scores $s_t^{(k)}$,

and the deployed mask is $\hat{m}_t^\star = \hat{m}_t^{(k^\star)}$ with $k^\star = \arg\max_k s_t^{(k)}$. This score also gates whether the frame's embedding is written into the memory bank, preventing low-quality frames from corrupting propagation.

This step replaces what would otherwise be either manual frame-by-frame mask annotation or a dedicated banner-detection model trained on tournament-specific data.

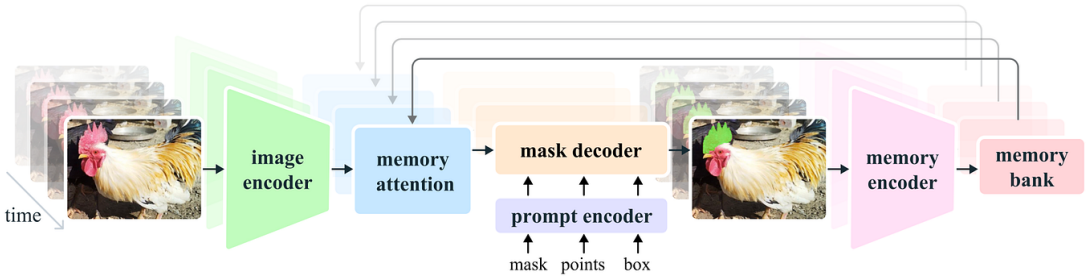

Figure 2: Architecture of SAM 2, reproduced from [4]. A Hiera image encoder feeds frame embeddings into a memory-attention block whose keys and values come from a memory bank of recent and prompted frames; the mask decoder takes the conditioned features together with prompt-encoder tokens and emits the per-frame mask. The memory encoder writes each output back into the memory bank for future frames. This is the design that turns a single click on one frame into a temporally coherent mask track across hundreds of frames.

4.3 Hull-based quad fitting

Each per-frame binary mask is reduced to a four-corner placement quadrilateral via convex-hull vertex deduction. We compute the convex hull $\mathcal{H}_t$ of the mask, then select four corner points that maximise perpendicular distance from each candidate edge. The hull fitter tolerates partial occlusion (a banner whose mask extends off-screen) better than principal-component or linear-programming alternatives. Exponential-moving-average smoothing with a small temporal coefficient keeps the four corners stable frame-to-frame.

4.4 Court-keypoint detection and homography initialisation

The court-geometry estimator is the dynamic component of the placement. It computes a per-frame homography that maps the canonical court plane to the broadcast image plane.

We adapt the heatmap-regression convolutional network of TrackNet [5], following the open-source `TennisCourtDetector` adaptation [6] that regresses fourteen semantically labelled court landmarks (four baseline corners, four singles-line intersections, four service-line corners, and two T-points at the centre service line). The model is a fully convolutional encoder-decoder: a VGG-style contracting path of three pooling stages with 64, 128, 256 channels, followed by a symmetric expanding path with bilinear up-sampling, producing a fifteen-channel output tensor at the input resolution of 640×360 pixels (fourteen keypoint channels and one auxiliary channel). Each output channel is passed through a sigmoid activation to yield a per-pixel confidence map for one court landmark:

$$\hat{Y}_k(u, v) \;=\; \sigma\big(\, f_\theta(I_t)\big)_{k,u,v}, \qquad k = 1, \ldots, 14. \tag{2}$$

Sub-pixel keypoints are extracted by thresholding each heatmap at $\tau = 170/255$ and locating the centre of the strongest connected blob using OpenCV's `HoughCircles` with a very loose accumulator threshold (`param2 = 2`) and radii in $[10, 25]$ px on the 640×360 resized frame. This is a peak-centring trick inherited from the upstream `TennisCourtDetector` implementation: HoughCircles is normally a circle detector, but the loose `param2` and the binarised input cause it to return the centre of the strongest super-threshold blob. Concretely,

$$(\hat{u}_k, \hat{v}_k) \;=\; \mathrm{peakCentre}\big(\mathbf{1}\{\hat{Y}_k > \tau\}\big).$$

Given $|\mathcal{J}| \geq 5$ image-reference correspondences against a canonical metric court template, we estimate the court-to-image homography H_t^{net} by RANSAC [11] with a five-pixel reprojection threshold. RANSAC selects the homography that maximises the consensus-set size

$$\hat{H}_t^{\mathrm{net}} \;=\; \arg\max_{H} \Big|\big\{\, k \in \mathcal{J} \;:\; \big\|(\hat{u}_k, \hat{v}_k) - \pi\big(H\,[X_k, Y_k, 1]^\top\big)\big\|_2 < \tau_{\mathrm{reproj}} \,\big\}\Big|, \quad \tau_{\mathrm{reproj}} = 5 \text{ px}, \tag{3}$$

where π is the perspective projection; the returned H_t^{net} is then refined by least-squares (DLT) over the inlier consensus set. When at least five keypoints are detected we run the RANSAC path above; with exactly four keypoints we fall back to an exhaustive best-of-twelve four-point configuration search (inherited from [6]); with fewer than four detections the estimator returns no homography on that frame and the placement holds the seed.

To align the learned-keypoint reference frame with the project's pre-existing classical-line backend (so that the YAML-configured `court_quad` fractional coordinates resolve to the same image-plane locations under either estimator), we compose a one-shot bridge between the two reference frames on frame zero:

$$B \;=\; H_{\mathrm{classical}}(0)\, H_0^{\mathrm{net}\,-1}, \tag{4}$$

and report the absolute court-plane homography as $H_t = B\, H_t^{\mathrm{net}}$ for all subsequent frames. The manually-clicked seed corners enter separately, via the hybrid-lock state machine described next; this bridge anchors the learned per-frame geometry to a consistent reference frame, not to the manually-clicked seed.

4.5 Hybrid-lock state machine

The hybrid-lock is a small state machine that runs after every keypoint-estimator call. It maintains the seed homography $H_{\mathrm{seed}} = H_{\mathrm{manual}}(0)$ and a per-corner *committed position* state $\{\mathbf{c}_i^{(\mathrm{out})}\}_{i=1}^4$. Per frame, letting $\mathbf{c}_i$ denote the i-th manually-clicked corner of a placement region, the gate computes the mean projected-corner displacement under the new homography:

$$d_t \;=\; \frac{1}{4} \sum_{i=1}^{4} \big\|\pi(H_{\mathrm{seed}}\, \mathbf{c}_i) - \pi(H_t\, \mathbf{c}_i)\big\|_2. \tag{5}$$

When $d_t < \tau_{\mathrm{lock}}$ the gate stays locked and the committed positions are held at $\pi(H_{\mathrm{seed}}\mathbf{c}_i)$. When $d_t \geq \tau_{\mathrm{lock}}$ the gate fires and each committed corner is iteratively re-aimed at the new target $\pi(H_t\mathbf{c}_i)$ over n frames via a per-corner recurrence,

$$\mathbf{c}_i^{(\text{out},k)} \;=\; \mathbf{c}_i^{(\text{out},k-1)} \;+\; \frac{1}{n-k+1}\,\big(\pi(H_t\mathbf{c}_i) \;-\; \mathbf{c}_i^{(\text{out},k-1)}\big), \quad k = 1, \ldots, n, \tag{6}$$

with ramp length $n = \max(R_{\min}, \lceil d_t / v_{\max} \rceil)$, $R_{\min} = 3$, $v_{\max} = 2$ px/frame. The output homography $H_t^{(\text{out})}$ is reconstructed at each frame by fitting a DLT homography from the canonical corners $\{\mathbf{c}_i\}$ to the current committed positions $\{\mathbf{c}_i^{(\text{out},k)}\}$. Our final configuration uses $\tau_{\text{lock}} = 30$ px: loose enough that keypoint-detection noise does not fire the gate spuriously, tight enough that real camera motion crosses the threshold and the dynamic estimate engages. The ramp length is adaptive: small displacements ramp in three frames; aggressive motion (large d_t) ramps over more frames so that the per-frame motion remains bounded by $v_{\max}$ pixels. This prevents pop artefacts when the gate fires.

On the Melbourne clip the eval-framework counters report 751 of 767 frames locked at H_{seed} (visually pixel-identical to the manually-clicked baseline), 16 frames inside the walkover window ramping toward the dynamic estimate, and 0 frames reaching the full estimate state (the camera motion subsides before any ramp completes). Without the hybrid lock the placement would either drift during the walkover (always-locked) or pulse with per-frame keypoint noise during the static portion (always-dynamic).

4.6 Player matte: MatAnyone

To handle player occlusion across the walkover window we require a continuous-valued alpha matte (not a bounding box, not a binary segmentation). We use MatAnyone [7], a 2025 video matting framework that predicts a per-pixel alpha matte $\alpha_t \in [0,1]^{H \times W}$ for each frame and propagates a single annotated first-frame target through subsequent frames via memory-augmented recurrent decoding.

MatAnyone extends the memory-based video object segmentation paradigm of Cutie [12] by storing previously predicted alpha mattes (rather than binary masks) in an external memory bank that is queried at each step via attention over query-key affinities. The single annotated first-frame matte seeds the memory and acts as the target assignment; subsequent frames are processed recurrently, with the value encoder writing the model's own alpha prediction back into memory to enable long-range temporal propagation. The central architectural contribution is a region-adaptive memory fusion module that mitigates the trade-off between temporal stability in semantically homogeneous core regions and responsiveness at fine object boundaries.

A lightweight three-layer convolutional head predicts a per-pixel change probability $U_t \in [0,1]^{H \times W}$ that gates the contribution of the previous-frame memory readout against the current-frame memory query. The region-adaptive fusion rule is

$$P_t \;=\; U_t \odot V_t^m \;+\; (1 - U_t) \odot V_{t-1}, \tag{7}$$

where $V_t^m = A_t V_m$ is the value read from the memory bank via the attention affinity A_t, V_{t-1} is the value propagated from the previous frame, and $\odot$ denotes the Hadamard product. U_t is supervised by the binarised alpha-difference indicator

$$U_t^{\text{GT}} \;=\; \mathbf{1}\big[\,\big|\alpha_{t-1}^{\text{GT}} - \alpha_t^{\text{GT}}\big| \geq \delta\,\big]$$

with $\delta = 10^{-3}$. Intuitively, U_t is the per-pixel change probability: $U_t \to 1$ in regions where the alpha changed substantially between frames (typically object boundaries), and the fusion weights the freshly-attended memory readout V_t^m so that fine detail is preserved; $U_t \to 0$ in stable core regions, and the fusion weights the propagated value V_{t-1} so that flicker is suppressed. Training combines pixel-wise L_1 loss in core regions with a Deep Decomposition Constraint loss along boundaries [7].

Binary segmentation (or a bounding box) was rejected because the player's silhouette at walkover frames is intricate (legs, racket arm, knee bend) and any hard cut produces visible vertical stripes through the floor logo. Alpha matting is the right primitive for this occlusion problem.

4.7 Inpaint plus LED-style compositor

Per frame, per placed region, the compositor performs four operations.

(1) Erase the original ad. Inside each placement quad we identify the text-mask region (the baked-in ad pixels) and replace it with a smooth fill matched to the surrounding court colour, plus matched-noise injection so the result does not read as a flat patch:

$$\hat{I}_t^{\text{inpaint}}(p) \;=\; \begin{cases} (G_\sigma * \text{fill}(I_t, \bar{c}_t))(p) \;+\; \eta(p) & \text{if } p \in \text{text\_mask}_t \\ I_t(p) & \text{otherwise,} \end{cases}$$

where $\bar{c}_t = \text{med}_{q \in \text{surround}(\text{text\_mask}_t)}\, I_t(q)$ is the spatial median of the surround region of the current frame, G_σ is a Gaussian blur with kernel size set by `shade_blur_ksize`, and $\eta(p) \sim \mathcal{N}(0, \hat{\sigma}^2_{\text{surround}})$ is texture-matched noise calibrated to the variance of the surround. A clean-court reference video, produced offline by temporal-median accumulation across player-absent intervals, is used downstream by the evaluation framework (walkover-window detection and occlusion-IoU) rather than by the per-frame compositor.

(2) Warp the new logo. The sponsor logo L is warped into the placement quad through the chosen homography:

$$\tilde{L}_t \;=\; \text{warp}\big(L,\; H_t^{\text{out}}\big).$$

(3) LED-style brightness re-bake. For each pixel inside the placement quad, the warped logo's chrominance is preserved while its luminance is re-baked against the local surface luminance computed from the inpainted background. Let Y denote the luminance channel. We compute a per-pixel shade map $s_t(p)$ that compares the Gaussian-blurred background luminance at p to the median background luminance inside the placement quad, raised to a configurable strength s and clipped:

$$\tilde{L}_t^{\text{led}}(p) \;=\; \tilde{L}_t(p) \cdot \text{clip}\left(\frac{Y\big(G_\sigma * \hat{I}_t^{\text{inpaint}}\big)(p)}{\bar{Y}_{\text{quad},t}} \right)^{s}, \qquad \bar{Y}_{\text{quad},t} = \text{med}_{q \in \text{quad}_t}\, Y\big(G_\sigma * \hat{I}_t^{\text{inpaint}}\big)(q), \quad (8)$$

with the gain clipped to $[\texttt{shade\_clip\_min}, \texttt{shade\_clip\_max}]$, σ set by `shade_blur_ksize`, and s set by `shade_strength`. This makes the inserted logo read like a physically painted or printed ad whose brightness tracks the local lighting on the surface: a logo placed in a shaded part of the court

darkens, a logo in a bright patch brightens, all without altering the logo's intrinsic colours. Without this step the logo reads as a 2D overlay floating on top of the frame.

(4) Person-matte occlusion. The final composite uses the MatAnyone alpha matte to occlude logo pixels behind the player:

$$O_t(p) \;=\; (1 - \alpha_t(p)) \cdot \tilde{L}_t^{\text{led}}(p) \;+\; \alpha_t(p) \cdot I_t(p), \tag{9}$$

where $\alpha_t(p) = 0$ marks pure background (the logo is fully visible) and $\alpha_t(p) = 1$ marks pure foreground (the player covers the logo). Continuous values in between handle the silhouette anti-aliasing that binary segmentation cannot.

The compositor additionally supports an optional shadow-synthesis mode (multiplying the placed pixels by a Gaussian-blurred dilation of the player mask, producing a soft player-foot cast shadow on a floor logo). This was disabled in the final configuration following the visual-review override described in Section 6.4.

Figure 3: Compositor stage walkthrough on a single banner. Left to right: (1) the original broadcast crop containing the baked-in *blua.* ad; (2) the placement quad detected and outlined; (3) the quad rectified to a fronto-parallel rectangle in the canonical surface frame; (4) the replacement *Ferrari* logo composed in the same rectified frame; (5) the new logo warped back through the inverse homography into the original perspective and blended via the LED-style luminance re-bake (Equation 8). The same pipeline runs per frame for every placement region.

5 Evaluation Framework

A central deliverable of this project is the evaluation framework we built to score candidate runs. It has three layers, each capturing a different failure mode.

5.1 Three layers

Layer 1: deterministic numerical metrics. Per-region scorecards with hard gates. Catches geometric instability, temporal flicker, and walkover-occlusion failures. Fast, reproducible, gates the build.

Layer 2: structured visual rubric. A thirteen-dimension rubric scored manually by a reviewer against paired top-original / bottom-composite crop strips, with the original baked-in ad as the comparison anchor. Catches qualitative regressions that numerical metrics miss (halo, edge reflex, texture match, player contact shadow).

Layer 3: direct visual review. Human review of the output video against the original broadcast. The accept-or-reject vote; tie-breaks the numerical and rubric layers when they disagree.

5.2 Layer 1: numerical metrics

Each candidate run is scored by the eval framework. The framework reads the run's composited video plus its frozen configuration, auto-discovers placement regions, and computes per-region

Table 2: Hard gates in the eval framework. Each region must pass all applicable gates to ship; the two walkover gates apply only to the court-floor region. C_t denotes the clean-plate frame and L_t^{baked} denotes the gold composite's logo at the same frame index. The SSIM definition follows [13].

Metric	Symbol / definition	Gate	What it catches
`corner_max_jump_px`	$\max_t \lVert \mathbf{c}_t - \mathbf{c}_{t-1} \rVert_\infty$	< 2.0	single-frame placement jump
`corner_accel_p95_px`	$\text{pct}_{95} \lVert \mathbf{c}_{t+1} - 2\mathbf{c}_t + \mathbf{c}_{t-1} \rVert$	< 1.0	jittery corner trajectory
`quad_area_cv`	$\sigma(A_t)/\mu(A_t)$, $A_t = \text{area}(\text{quad}_t)$	< 0.05	placement-area flicker
`roi_jitter_ratio`	$\frac{\text{frame-diff}_{\text{ROI}}^{\text{comp}}}{\text{frame-diff}_{\text{ROI}}^{\text{orig}}}$	≤ 1.05	ROI instability vs original
`roi_temporal_ssim_mean`	$\text{mean}_t \, \text{SSIM}(\text{ROI}_t, \text{ROI}_{t-1})$	> 0.95	temporal coherence in ROI
`walkover_logo_visible_pct`	fraction of quad_t pixels with logo signal in player-absent rows	> 0.10	logo over-erased during walkover
`walkover_occlusion_iou`	IoU of $\mathbf{1}\{\lvert Y(I_t) - Y(C_t)\rvert > T\}$ vs $\mathbf{1}\{\lvert Y(O_t) - Y(L_t^{\text{baked}})\rvert > T\}$ inside quad_t, with $T = 20$ on 8-bit luminance	> 0.80	occlusion accuracy in walkover window

metrics. Hard gates are listed in Table 2.

A secondary set of warning-only metrics surfaces issues without gating the build: Lab ΔE colour match against a neighbouring same-surface patch, noise-variance ratio (warns when the placed region is noticeably smoother than its neighbour, i.e., the "pasted-on" failure mode), and edge-sharpness ratio (warns when the placement-quad boundary has materially higher gradient magnitude than the rest of the frame).

The framework also produces reference-comparison metrics when a gold run is configured. A 5% deviation in the wrong direction flips an `any_regression: true` flag and the exit code to 3, even when per-region gates all pass. The reference comparison computes corner distance versus gold, region SSIM versus gold, and walkover-IoU delta versus gold.

5.3 Layer 2: structured visual rubric

Numerical metrics catch motion and structural problems but miss perceptual "does this look right" failures. Layer 2 adds a structured visual rubric scored by a human reviewer against paired crop strips.

For each candidate run the eval framework writes paired strips per region: the top row is the unmodified original broadcast (the real baked-in ad), the bottom row is our composite at the same frames. The original is always present as the comparison anchor, because absolute scoring drifts toward "looks fine" without it.

The reviewer assigns integer scores in $\{1, \ldots, 5\}$ across thirteen dimensions per region:

- **Realism**: painted-on vs pasted-on, edge seam visibility, texture match, halo presence, edge reflex.
- **Colour**: hue match, brightness match, saturation match.
- **Geometry**: perspective plausibility, size plausibility.
- **Temporal** (walkover/floor only): occlusion realism, jitter visibility, player contact-shadow plausibility.

The rubric scores are surfaced alongside Layer 1 metrics in the run's machine-readable quality summary. Rubric scores are warning-only; they never gate the build, but they guide where to investigate.

5.4 Layer 3: direct visual review

The final gate is direct human review of the output video against the original broadcast. This is the accept-or-reject vote, and it overrides the lower layers in cases of disagreement.

The reason for this third layer is that Layer 1 and Layer 2 can both miss the same class of failure: a candidate run might pass every numerical gate, score well on the rubric, and still produce a composite that any viewer would immediately read as fake. The lesson from Phase 3 (Section 6.4) is that this is not a hypothetical; it actually happened to us on a high-scoring candidate.

5.5 Walkover-window detection

The floor-logo walkover-specific metrics require knowing which frames the player is on the logo. We auto-detect the walkover window:

1. Pad the floor placement quad's bounding box by 30 px horizontal × 60 px vertical.
2. Compute the per-frame mean absolute colour delta between the original video and a clean-court reference video, restricted to the padded quad and averaged over BGR channels: $\Delta_t = \text{mean}_{p \in \text{padded quad}} \frac{1}{3} \|I_t(p) - C_t(p)\|_1$.
3. Box-smooth $\{\Delta_t\}$ with a five-frame kernel.
4. Threshold at $\mu(\Delta) + 2\,\sigma(\Delta)$, take the longest contiguous super-threshold run, pad ±10 frames.

On the Melbourne clip this auto-detects frames 685 to 723 as the walkover window, which matches manual annotation.

6 Project Journey

The final design was reached through three iteration phases. We document them because the negative result from Phase 2 narrowed the design space and the Phase 3 visual-review override is informative about how to use numerical and rubric scores in practice.

6.1 Phase 1: V68, manually-clicked corners

Goal. An end-to-end working pipeline producing a watchable composite on the Melbourne clip.

We manually clicked the four court corners and the four corners of each of the five placement regions on a single seed frame. The resulting homography was locked across the entire clip. The compositor settings (mask dilation, alpha feather, inpaint method, local colour matching, blend mode) were hand-tuned to this configuration (designated **V68**) and persisted unchanged into the final deliverable.

Result. Visually excellent on the static portions of the clip; failed any time the camera moved, with logos visibly drifting off the court. This was the binding limitation that motivated Phase 2.

V68's per-region scorecard (held as the regression-gold reference for the rest of the project) passes every gate by construction: floor_walkover_occlusion_iou = 1.000 (it is itself the gold), temporal SSIM ≥ 0.997 across regions.

6.2 Phase 2: hybrid-lock with a line-based estimator (failed axis)

Hypothesis. Replace the static homography with a per-frame `classical_lines_v1` estimator (Canny edges, probabilistic Hough lines, RANSAC over candidate line intersections), gated by a hybrid-lock state machine so noisy frames stay at the seed.

We swept the tolerance $\tau_{\text{lock}} \in \{2, 4, 6, 10, 15, 30, 99999\}$ and ramp speed $\in \{0.3, 1.0, 2.0\}$ px/frame over two waves of parallel runs on H200 GPUs. The result is unambiguous (Table 3).

Table 3: Phase 2 tolerance sweep with the line-based estimator. Floor SSIM regresses monotonically as the tolerance loosens; only the always-locked sanity baseline passes all gates.

τ_{lock} (px)	Floor ROI SSIM	Back ROI SSIM
4	0.21	0.99
6	0.39	0.99
10	0.59	0.99
15	0.76	0.99
30	0.85	0.99
99999 (sanity, V68 static)	0.9996	0.9999

Diagnosis. The line-based estimator is frame-to-frame noisy, not frame-zero misaligned. Even with the calibrated court-plane reference, per-frame projected corners deviated from the seed enough to fire the ramp gate on a substantial fraction of frames; heavy EMA smoothing did not rescue tight tolerances. We note one confounder: the displacement gate of Equation 5 is the mean L2 displacement across the four projected corners, which is direction-blind, so small but systematic vertical drift can be absorbed into the gate without firing. An axis-decomposed gate (separate vertical and horizontal thresholds) was not tested. With that caveat, the actionable conclusion of the sweep is that for this estimator and this gate, no Pareto improvement over the always-locked baseline was reachable; the path forward was a more stable upstream estimator (Phase 3) rather than further controller tuning.

Decision. Hold V68 as the regression gold. Pivot the dynamic-homography axis to a more stable estimator (Phase 3).

6.3 Phase 3: TrackNet-derived port and iteration

We replaced `classical_lines_v1` with the TrackNet-derived court-keypoint detector described in Section 4.4, composed with the frame-zero bridge of Equation 4, and ran the hybrid-lock with $\tau_{\text{lock}} = 30$ px. The first run with the new backend, designated **P3-A1**, was sufficient: floor-region temporal SSIM 0.9927, walkover occlusion IoU 0.985, every gate passes. P3-A1 became the baseline for the iteration phase that followed.

After P3-A1 we ran approximately fifty further iteration cycles on H200, sweeping compositor parameters: shadow synthesis strength, banner-edge feather, inpaint method, padding fractions, and so on. Each cycle produced one branched configuration and one inference run; metrics and rubric scores were collected centrally. The iteration explored:

- Feather and dilation fine-tuning on V68's compositor.
- Contact-shadow root cause (which turned out to be the `erase_text` flag on the court-floor surface, not the occlusion-mask dilation we initially suspected).

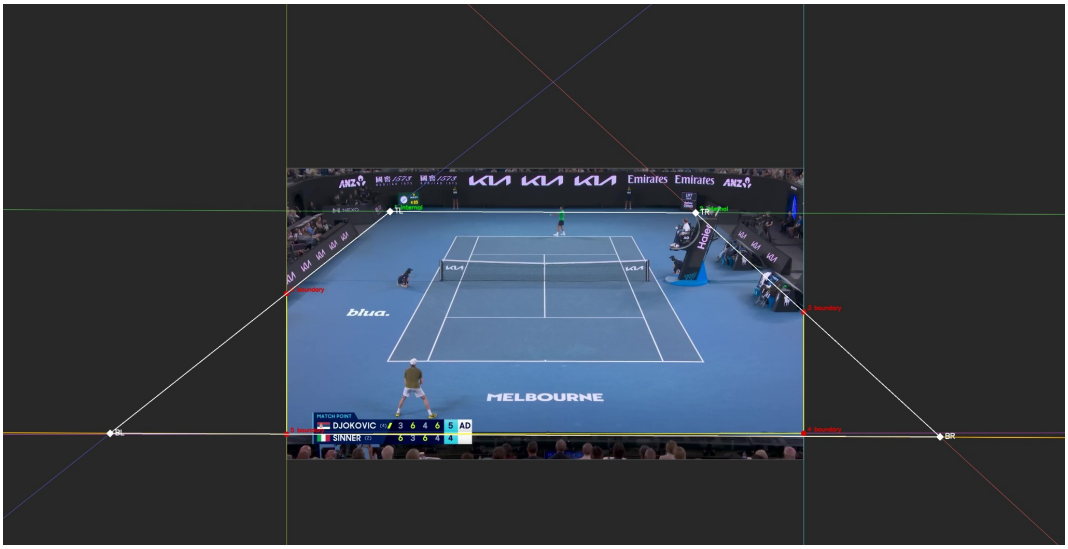

Figure 4: The classical line-based estimator on the demonstration clip. Hough lines are detected on the court markings (yellow / red), classified by orientation into a depth family and a width family, and intersected to recover the depth vanishing point (off-frame to the left and right). The estimator is unbiased on a static camera in expectation, but the per-frame intersection point drifts by 5 to 15 pixels between consecutive frames because the underlying line set is itself noisy. That residual is large enough to fire the hybrid-lock ramp gate on a substantial fraction of frames and small enough that EMA smoothing cannot suppress it without introducing latency.

- A code change introducing **shadow synthesis**: multiplying inserted logo pixels by a Gaussian-blurred dilation of the player mask, to synthesise a soft player-foot cast shadow.
- A shadow-strength sweep that identified 0.6 as the photographically credible setting (0.3 to 0.4 reads as too soft; 0.7+ as a "blob").

The candidate that won the structured visual rubric on the two user-flagged artefact dimensions (`halo_presence` and `edge_reflex`) was **P3-A38/e2**: a recipe combining P3-A1's TrackNet-plus-hybrid-lock with shadow synthesis at strength 0.6, the `erase_text` flag on the court floor, and a tightened banner edge.

6.4 The visual-review override

P3-A38/e2 scored 5/5 on `realism.halo_presence` and `realism.edge_reflex` (the two dimensions a stakeholder reviewer had previously flagged on V68) and passed every Layer 1 gate. On the numerical gates and the rubric alone, it was the recommended candidate.

On direct visual review (Layer 3) it had visible regressions versus P3-A1:

- The synthesised contact shadow darkened logo pixels under the player's feet in a way that read as a darkened blob rather than a shadow.
- `erase_text=true` removed the painted MELBOURNE wordmark from under the floor logo. The rubric counted this as "no bleed-through," which it technically is, but visually the logo now sat on a plain green floor instead of the patterned painted court area, which read as artificial.
- Tightened banner padding exposed harder banner edges on the left logo. The rubric called

Table 4: Per-region quantitative metrics for the final deliverable (P3-A1). Gates: SSIM > 0.95, jitter ≤ 1.05, walkover visible > 0.10, walkover IoU > 0.80.

Region	Pass	Temporal SSIM	Jitter ratio	Walkover visible %	Walkover IoU
Back banners		0.9999	0.291	n/a	n/a
Left-side banner		1.0000	0.390	n/a	n/a
Floor logo		0.9927	0.805	0.179	0.985
Full frame		0.9987	0.687	n/a	n/a

this `edge_reflex=5` (no smearing); on direct viewing the harder edge read as "pasted on" more than the slightly-softer P3-A1 baseline did.

The probable cause is that the rubric is scored in absolute terms (1 to 5 per dimension), not as a direct comparison against the original baked-in ad in the same broadcast frame. Without that anchor present at scoring time, the reviewer's scale drifts toward "looks fine" for any variant that looks remotely competent. The pairing-based prompt was specified in our reviewer brief but turned out not to be sufficient to enforce direct-comparison scoring discipline.

Lesson. A numerical rubric, however thoughtfully designed, is not a substitute for direct human visual review against ground truth. Layer 1 gates and Layer 2 rubric scores are excellent regression detectors, but the final accept-or-reject decision needs a human looking at the actual video against the original broadcast.

Decision. P3-A1, the TrackNet-port baseline before any compositor tweaks, was designated the final deliverable. The shadow-synthesis code and the v2 rubric both remain on the production branch behind feature flags so future work can revisit those knobs if it wishes; the final configuration does not enable them.

7 Results

7.1 Final deliverable

The final delivered configuration is **P3-A1**, with the recipe:

- V68 manually-clicked court corners as the seed homography H_{seed}.
- TrackNet-derived court-keypoint detector for the per-frame homography H_t^{net}.
- Hybrid lock at $\tau_{\text{lock}} = 30$ px, three-frame ramp.
- V68's compositor settings, unchanged: median-fill inpaint, alpha feathering, local colour matching, LED-style brightness re-bake. No shadow synthesis. No `erase_text`. No tightened banner padding.

7.2 Quantitative results

Table 4 reports per-region results extracted from the eval framework's machine-readable summary. All region scorecards pass (the back banners are reported as a group; the left-side banner, the floor logo, and the full-frame summary are reported separately), and the walkover-window evaluation passes both floor-specific gates.

Reading the metrics. Temporal SSIM at 0.9999 on the back banners means the placement is,

frame to frame, visually identical to the V68 manually-clicked gold (the small residual deviation reflects only the inpaint's frame-to-frame variance). The left side banner reads identically (SSIM 1.0000 at four decimal places). The floor logo's slightly lower SSIM at 0.9927 is the expected consequence of two factors: the active hybrid lock during the walkover window introduces a few frames of correctly-tracked motion, and the floor surface has more luminance heterogeneity than the dark back banners (so any compositor noise is more visible).

The walkover occlusion IoU at 0.985 is the most important single number on this table. The metric measures, inside the floor placement quad, the IoU between *where the player actually is* (estimated as the threshold of $|I_t - C_t|$ against the clean plate) and *where the composite hides the logo* (estimated as $|O_t - L_t^{\text{baked}}|$ against the gold's baked logo). A perfect occlusion would score 1.000; our gate is at 0.80. We achieve 0.985, meaning that for 98.5% of the area where the player is visibly present, our composite correctly suppresses the logo, and conversely 98.5% of the area where the composite suppresses the logo coincides with the player's actual silhouette. The remaining 1.5% is split between sub-pixel matte edges (anti-aliasing at the player silhouette boundary) and a small number of frames where the MatAnyone matte is slightly conservative around the racket arm.

The walkover *visible* percentage of 0.179 means that 17.9% of pixels inside the floor placement quad show logo signal in player-absent regions of the frame. The gate requires > 0.10; the gold's value is 0.179 (we match by design). A low value here would indicate that the inpaint had over-erased.

The `any_regression: true` flag is set against V68, driven by a single metric, the floor-region jitter-ratio regression flag (the `regression_*` mirror of `floor_roi_jitter_ratio` in the eval framework), which moves from 0.494 (gold) to 0.805 (P3-A1), a 63% increase. **This is by design**: V68 is statically locked, so its floor jitter is the lowest physically possible (frame-to-frame inpaint variance only). P3-A1's hybrid lock introduces a small amount of correctly-tracked motion in the walkover window, which the jitter metric flags as a deviation. The visual outcome is the desired one (the placement now follows the camera), so the flag is treated as an expected side effect, not as a regression.

7.3 Visual results

Figure 5 shows the back-banner crop strip: three banner positions on the back wall, paired top-original / bottom-composite.

Figure 6 shows the left-side banner.

Figure 7 shows the court-floor logo during the walkover window. Three frames are captured at $f694$ (pre-contact), $f704$ (player directly on the logo), and $f713$ (post-contact), each paired top-original / bottom-composite.

Figure 8 shows the per-frame forensic decomposition across the entire walkover window. Each row is one of the five key frames in the auto-detected window (entry, pre-contact, contact, post-contact, exit). The six columns in each row are, left to right: the original broadcast crop; the clean court reference (no logo, no player); our composite; the absolute difference between original and clean (which highlights the player's pixel footprint, brighter where the player physically blocks the wordmark); a *survival* heatmap of inserted-logo pixels surviving the alpha-matte gate (red where the logo remains visible, dark blue where the player covers it); and a red leak overlay marking pixels suspected of mis-occlusion.

A correctly behaving pipeline produces a characteristic signature across the grid. In the original-

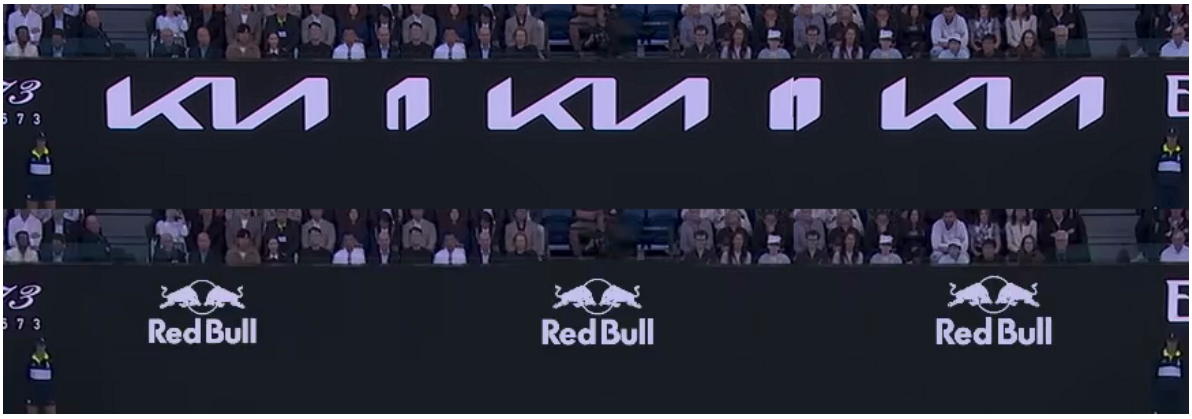

Figure 5: Back-wall banners at three distinct placement positions (objects 1, 2, 5), captured at the same frame index. Top row: original broadcast with the baked KIA ad. Bottom row: our composite with the inserted Red Bull logo.

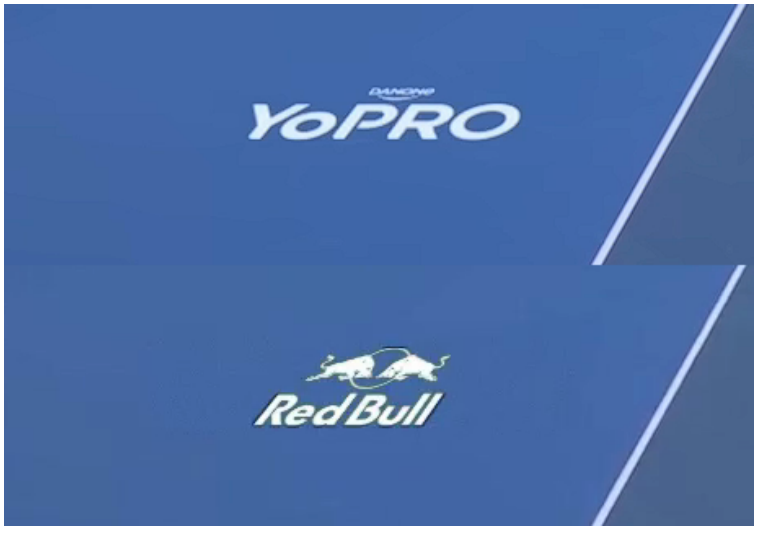

Figure 6: Left side banner. Top: original broadcast with the baked YoPRO ad. Bottom: our composite with the Red Bull logo. The mild halo at letter edges is the residual texture-match limitation.

Figure 7: Court-floor logo during the walkover window. Three frames inside the auto-detected window of frames 685 to 723. Top row: original broadcast with the painted MELBOURNE wordmark. Bottom row: our composite with the inserted Red Bull logo, occluded correctly through the player's silhouette via the MatAnyone alpha matte.

minus-clean column (4), the player's silhouette is sharp and bright where they cover the wordmark and dark everywhere else; this is the ground-truth "where the player is" signal. In the survival column (5), the inserted Red Bull logo appears as bright red signal in the regions the player does *not* occupy, and is suppressed (dark blue) wherever the player covers it; this is "where the logo is visible." The red leak overlay (column 6) should be near-empty: a successful occlusion produces only sub-pixel anti-aliasing artefacts at the silhouette boundary. Reading down the rows tells the temporal story: at entry and exit the logo is fully visible and uncovered, at pre-contact and post-contact the player covers the wordmark progressively, and at the contact frame (row 3, $f704$) the player is directly atop the logo and the survival map shows the clean silhouette cutout. The walkover occlusion IoU of 0.985 reported in Table 4 is the IoU between columns 4 and 5 aggregated across all 39 frames in the window.

A side-by-side regression video stacks our composite, the V68 gold composite, and an absolute-difference heatmap into a single 2868 $\times$ 536 frame for scrubbing. Bright pixels in the heatmap denote larger per-pixel deviations from gold; on a successful candidate the heatmap is mostly dark.

7.4 Throughput and resource use

End-to-end inference runs at **2.68 fps** on a single NVIDIA H200 GPU for the 767-frame demonstration clip (286 seconds of wall-clock time, 139.8 GB of peak VRAM used). The throughput is dominated by SAM 2's video tracker (memory attention is the per-frame hot path) and the MatAnyone alpha-matte forward pass, both of which run at native resolution on every frame. The court-keypoint detector and the compositor stages are comparatively cheap. Table 5 summarises the on-demand GPU options on the cloud platform used during iteration and their hourly cost; the project's iteration phase used the H200 for its memory headroom (the alpha-matting forward pass exceeds the working memory of smaller A-class GPUs on 1920-px frames) and ran approximately 50 cycles in Phase 3.

Figure 8: Walkover sequence forensic decomposition. Rows top to bottom: $f685$ (entry, player begins entering), $f694$ (pre-contact, approximately 25% across the window), $f704$ (contact, player directly atop the logo, the showpiece frame), $f713$ (post-contact, approximately 75% across), $f723$ (exit, window closes). Columns left to right: original broadcast; clean court reference; our composite; original-minus-clean delta; logo-survival heatmap; suspected-leak red overlay. The composite column (3) shows the inserted Red Bull logo correctly occluded through the player's full silhouette across the entire window.

Table 5: On-demand GPU options used during iteration, with VRAM and per-hour cost (USD). The final run used a single H200 for 286 s, an end-to-end cost of approximately 0.36 USD per clip at on-demand pricing.

GPU	VRAM	USD/hr
T4	16 GB	0.59
L4	24 GB	0.80
A10G	24 GB	1.10
L40S	48 GB	1.95
A100	40 GB	2.10
A100-80GB	80 GB	2.50
H100	80 GB	3.95
H200 (final)	**141 GB**	**4.54**
B200	192 GB	6.25

The 2.68 fps figure is approximately one order of magnitude below the real-time broadcast target (approximately 30 fps). Closing that gap is an independent engineering axis discussed in Section 9: the pipeline is per-frame and each stage admits lower-precision inference, batched processing, or distillation to a smaller model without affecting the placement quality.

7.5 Headline numbers

For at-a-glance reference:

- **5** simultaneous virtual placements (three back banners, one left-side banner, one court-floor walkover logo).
- **0.9999** temporal SSIM on back banners (visually identical to V68 gold on locked frames).
- **0.985** walkover occlusion IoU (gate threshold > 0.80; gold is 1.000).
- **767** frames at approximately 59 fps; a 13-second demonstration clip.
- approximately **50** GPU iteration cycles in Phase 3.
- **13** rubric dimensions per region, scored across the five placement regions (three walkover-only dimensions apply to the floor logo, giving approximately 60 rubric cells per candidate run).
- **2.68 fps** end-to-end on H200 (real-time target approximately 30 fps is future work).

8 Discussion

8.1 Strengths

The pipeline is broadcast-credible on the static portions of the clip: visually indistinguishable from the V68 manually-clicked gold reference, which itself sets a high bar against the original baked-in ads. It is stable through the walkover window, with an occlusion IoU of 0.985 (against a gate of 0.80) and a per-pixel-accurate player matte from MatAnyone. It is software-only, with no in-venue hardware requirement. It needs only one to three clicks per region on a single seed frame plus a clean-plate reference video that can be produced offline.

The three-layer evaluation framework is reusable. Numerical gates and rubric dimensions both transfer to other tournaments via a simple configuration entry, and the walkover-window auto-detector generalises to any clip where a clean-court reference is available.

8.2 Limitations

Several quality limits remain.

- **Texture match on the left banner and floor logo.** Smoothed inpaint micro-grain is visible at close zoom against the gritty real court paint and banner material. Lifting this would require real texture transfer (noise injection, GAN-based inpaint, or a learned texture prior), not a parameter sweep on the existing compositor.
- **End-to-end throughput.** 2.68 fps on H200 is approximately one order of magnitude slower than the real-time target for live broadcast use. The pipeline was designed per-frame so that throughput is an orthogonal optimisation axis, but we did not pursue it in this phase.
- **Single-clip evaluation.** The pipeline was validated on one demonstration clip. The evaluation framework supports multi-clip operation, but only the Melbourne walkover is wired in at hand-off.
- **Calibration of metric thresholds.** Numerical gates were set by hand from the V68 baseline. With more clips a per-clip or learned threshold would be more robust. The Lab ΔE warning is dataset-specific and would benefit from re-derivation.

8.3 Lessons learned

Three observations that the project's documentation should pass forward.

For this estimator and this gate, per-frame estimator noise dominated the controller-side knobs. Phase 2 chased the dynamic-homography axis with a line-based estimator that was unbiased in expectation but noisy frame-to-frame. No setting of EMA smoothing, ramp design, or tolerance in the swept grid recovered a Pareto improvement over the always-locked baseline. We caveat that an axis-decomposed gate metric was not tested and could partially confound this conclusion; with that caveat, the path forward was a more stable upstream estimator (the TrackNet-derived backend used in Phase 3) rather than further controller tuning.

Pixel-perfect seed beats per-frame estimation alone. The frame-zero bridge that composes the keypoint detector's reference frame against the manually-clicked corners (Equation 4) is what makes the hybrid lock work. Without that bridge, per-frame estimates would live in their own coordinate frame and would not produce pixel-locked behaviour during the static portion. The combination wins; either piece alone does not.

Numerical rubrics are regression detectors, not accept-or-reject votes. The Phase 3 visual-review override was the most important methodological finding of the project. The candidate that won the rubric on the user-flagged dimensions and passed every numerical gate was, on direct viewing, worse than the simpler baseline. We adopted a three-layer evaluation explicitly so that Layer 3 (direct visual review) overrides Layers 1 and 2 in cases of disagreement.

9 Future Work

We list four directions in approximate order of expected payoff.

Texture transfer for inpaint and composite. Replace the smoothed median-fill inpaint with a learned-prior method (a diffusion-based or GAN-based inpaint trained on broadcast court textures). This is the single change most likely to lift the residual texture-match ceiling on the floor logo and the left banner.

Real-time throughput. A ten-fold speed-up is required for live broadcast. The pipeline is per-frame so the work decomposes naturally: SAM 2, the court-keypoint detector, the hybrid-lock state machine, MatAnyone, and the compositor. Each stage is independently amenable to lower-precision inference, batched processing, or distillation to a smaller model.

Multi-clip validation. Add at least two additional tournament clips (different surfaces, lighting, camera rigs) to the evaluation framework and validate that the pipeline's parameters transfer.

Auto-detection of placement regions. A parallel branch in our implementation explored using SAM 3 text prompts to detect ad regions automatically, removing the manual click step on the seed frame. An HSV histogram scene-change gate brings cost down to 3 to 4 fps on A100-80GB while preserving detection quality on static and slowly-changing footage. This is a credible follow-up direction for the authoring workflow.

10 Conclusion

We presented an end-to-end software pipeline for virtual advertising in tennis broadcasts, demonstrated on a thirteen-second sequence with five simultaneous placements and a player walkover through one of them. The pipeline combines three foundation models (SAM 2 for region segmentation, MatAnyone for player matting, and a TrackNet-derived 14-keypoint court detector) with two project-specific components: a hybrid-lock state machine that holds the placement pixel-locked to a manually-clicked seed when the camera is still and ramps to the dynamic estimate when motion exceeds tolerance, and a custom inpaint-plus-LED-blend compositor.

All five placement regions pass deterministic per-region quality gates, walkover-window occlusion IoU is 0.985, and temporal SSIM exceeds 0.99 in every region. We also presented the three-layer evaluation framework we built (numerical gates, structured visual rubric, direct visual review) and reported the visual-review override that selected the final deliverable. The lesson, that a structured rubric, however carefully designed, is not a substitute for direct human visual review against the original broadcast, is the most important methodological finding of the project.

Acknowledgements

This work was conducted as part of the Computational Science and Engineering and Data Science Capstone Programme (AC297r) at the Harvard John A. Paulson School of Engineering and Applied Sciences, sponsored by Mitsubishi Electric Innovation Center (MELIC).

The author team is split between Harvard and Politecnico di Milano. Raghav Punnam and Enrique Díaz de León Hicks are MS students in Computational Science and Engineering and Data Science at Harvard University. Giovanni Visi and Martina Maiorana are MS students in Computer Science and Engineering at Politecnico di Milano.

We thank our MELIC industry partners for the problem framing and the regular feedback that shaped the quality bar, and our Harvard programme advisors for the structural guidance that shaped the methodology.

References

[1] Supponor, "Supponor: Digital billboard replacement technology." Commercial product, https://supponor.com, 2024.

[2] uniqFEED AG, "uniqFEED: Personalized sports broadcasting platform." Commercial product, https://uniqfeed.com, 2024.

[3] Vizrt, "Viz Arena: Augmented-reality graphics for live sports." Commercial product, https://www.vizrt.com/products/viz-arena, 2024.

[4] N. Ravi *et al.*, "SAM 2: Segment anything in images and videos," in *International conference on learning representations (ICLR)*, 2025. Available: https://openreview.net/forum?id=Ha6RTeWMd0

[5] Y.-C. Huang, I.-N. Liao, C.-H. Chen, T.-U. İk, and W.-C. Peng, "TrackNet: A deep learning network for tracking high-speed and tiny objects in sports applications," in *Proceedings of the 16th IEEE international conference on advanced video and signal based surveillance (AVSS)*, 2019. doi: 10.1109/AVSS.2019.8909871.

[6] yastrebksv, "TennisCourtDetector: Deep learning network for detecting tennis-court keypoints." https://github.com/yastrebksv/TennisCourtDetector, 2023.

[7] P. Yang, S. Zhou, J. Zhao, Q. Tao, and C. C. Loy, "MatAnyone: Stable video matting with consistent memory propagation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2025. Available: https://arxiv.org/abs/2501.14677

[8] N. Homayounfar, S. Fidler, and R. Urtasun, "Sports field localization via deep structured models," in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2017.

[9] X. Nie, S. Chen, and R. Hamid, "A robust and efficient framework for sports-field registration," in *Proceedings of the IEEE/CVF winter conference on applications of computer vision (WACV)*, 2021.

[10] A. Kirillov *et al.*, "Segment anything," in *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)*, 2023.

[11] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[12] H. K. Cheng, S. W. Oh, B. Price, J.-Y. Lee, and A. G. Schwing, "Putting the object back into video object segmentation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[13] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.